

Denial-of-Service Attacks on LoRaWAN

Eef van Es

Open University of the Netherlands
Heerlen, The Netherlands
eef@van-es.org

Harald Vranken

Open University of the Netherlands
Heerlen, The Netherlands
Radboud University
Nijmegen, The Netherlands
Harald.Vranken@ou.nl

Arjen Hommersom

Open University of the Netherlands
Heerlen, The Netherlands
Radboud University
Nijmegen, The Netherlands
Arjen.Hommersom@ou.nl

ABSTRACT

LoRaWAN is the dominant protocol for communication in low-power Wide Area Networks in several European countries, and is being used increasingly in other parts of the world. We identified three vulnerabilities in the LoRaWAN protocol specification that can be used for launching Denial-of-Service (DoS) attacks against end-devices in a LoRaWAN network. We validated that these vulnerabilities can be exploited for DoS attacks by creating and simulating Coloured Petri Net models of relevant parts of the LoRaWAN protocol.

CCS CONCEPTS

• **Security and privacy** → **Mobile and wireless security; Denial-of-service attacks**; • **Networks** → **Mobile and wireless security; Denial-of-service attacks**;

KEYWORDS

LoRaWAN, Denial-of-Service, Internet-of-Things, Vulnerability

ACM Reference Format:

Eef van Es, Harald Vranken, and Arjen Hommersom. 2018. Denial-of-Service Attacks on LoRaWAN. In *ARES 2018: International Conference on Availability, Reliability and Security, August 27–30, 2018, Hamburg, Germany*. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3230833.3232804>

1 INTRODUCTION

LoRa is a proprietary long-range, low-power wireless technology [15]. LoRaWAN is an open Low-Power Wide Area Network (LPWAN) protocol specification built on top of the LoRa technology, developed by the LoRa Alliance [1]. The LoRa Alliance published the first official release of the LoRaWAN specification (v1.0) in October 2015 [8]. The most recent release is v1.1, published in October 2017 [11][2].

LPWAN protocols such as LoRaWAN are designed for connecting devices that need to send small amounts of data over a long range, while maintaining long battery life. These characteristics are suited for building Internet-of-Things (IoT) networks or Wireless Sensor Network (WSN) applications. In June 2016, the Netherlands became

the first country in the world to have a nationwide LoRaWAN network for IoT applications. Currently, LoRaWAN is widely available in many regions around the world.

In this paper, we report our findings of a security analysis on the LoRaWAN protocol specification, in which we focused on the possibilities to launch Denial-of-Service (DoS) attacks against end-devices in a LoRaWAN network. We specifically choose to study the protocol specification, because all implementations are based on this. We conducted our security analysis by selecting a number of well-known, generic (network) attack types (including eavesdropping, replay, DoS, cryptanalysis, and man-in-the-middle attacks), and exploring whether they are feasible in a LoRaWAN network according to the LoRaWAN specification. Our analysis identified three vulnerabilities in the LoRaWAN specification that allow DoS attacks, all exploiting eavesdropping and replay: a beaconing vulnerability, a downlink routing vulnerability, and an end-device activation vulnerability.

We validated our findings by creating formal models of relevant sections of the LoRaWAN protocol using Coloured Petri Nets (CPNs). We simulated and analysed our models with the tool CPNTools [5], which confirmed that these vulnerabilities indeed can be exploited in DoS attacks. We initially conducted our security analysis in the first half of 2017, based on release v1.0.2 of the LoRaWAN protocol specification as published in July 2016 [9][10]. When release v1.1 appeared in October 2017, we validated whether the DoS attacks that we identified still would be possible, which indeed is the case.

Although we originally discovered the vulnerabilities leading to DoS attacks independently of other researchers, these vulnerabilities recently have been published to some extent by other researchers [16][3][17][4]. However, these prior publications do not or only marginally describe how these vulnerabilities can be exploited for DoS attacks. The novelty of our work as described in this paper therefore is on how vulnerabilities, that are still present in the latest release of the LoRaWAN protocol specification, can be exploited for DoS attacks.

The remainder of this paper is organized as follows: In section 2 we briefly describe the security measures in the LoRaWAN protocol. In section 3 we present the three vulnerabilities that enable DoS attacks, and in section 4 we present the CPNs that actually demonstrate these attacks in simulation. We conclude the paper in section 5.

2 LORAWAN

The LoRaWAN architecture is based on a star-of-stars topology. The heart of the architecture is a network server which controls the network, and forms a bridge between the LoRaWAN and one or

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

ARES 2018, August 27–30, 2018, Hamburg, Germany

© 2018 Copyright held by the owner/author(s). Publication rights licensed to the Association for Computing Machinery.

ACM ISBN 978-1-4503-6448-5/18/08...\$15.00

<https://doi.org/10.1145/3230833.3232804>

more application environments. The network server uses standard TCP/IP links with one or more gateways. A gateway provides a bridging functionality between the network server at the back-end and end-devices at the front-end. A wireless communication link (single hop wireless LoRa Radio Frequency communication) is used between end-devices and one or more gateways.

An end-device is not associated with a specific gateway. The end-device broadcasts its data (uplink communication), which is typically received and relayed by one or more gateways. The network server will provide de-duplication of network packets. Depending on signal quality indicators of the last uplink communication, the network server decides and remembers which network path (gateway) is the best for downlink communication. Although bi-directional communication is possible, it is expected that uplink communication is the predominant type of traffic.

For security functionality, LoRaWAN uses the symmetric-key Advanced Encryption Standard (AES) [13] for encrypting, decrypting and message signing. For confidentiality, LoRaWAN (since v1.0.2) by default encrypts the payload in a message with the application security key (AppSKey). For integrity, all LoRaWAN messages (besides Join-Request messages and beacons) are signed with a Message Integrity Check (MIC) that is derived from a hash of the message content and a network security key (NwSKey). Both AppSKey and NwSKey are derived from a root key (AppKey) that is known by both the end-device and the network server. All keys are unique for each end-device. For availability, LoRaWAN supports multiple gateways (combined with packet de-duplication on the network server), and optionally message confirmation, which must be controlled by the application level.

3 VULNERABILITIES AND DOS ATTACKS

Our security analysis resulted in the discovery, or confirmation, of several vulnerabilities that can be exploited in DoS attacks against end-devices. We describe the details of these vulnerabilities in this section.

3.1 Beaconsing vulnerability

LoRaWAN supports different types of end-devices (Class A, B, and C), which provide a trade-off between battery lifetime and bi-directional communication options. Device Class A is default and consumes the lowest amount of energy. For Class A end-devices, each uplink transmission window is followed by two, fixed-length, downlink reception windows. Class B end-devices support the same transmit and reception windows as Class A end-devices, and additionally scheduled reception windows. These scheduled reception windows provide a mechanism for the network to contact the end-device at specific times rather than depending on the non-deterministic downlink reception windows available in the default Class A operation. These additional reception windows on Class B end-devices are achieved by gateways broadcasting beacons on a regular interval to synchronize the Class B end-devices in the network. Class B end-devices open additional reception windows based on a time reference in the broadcasted beacons. The LoRaWAN network uses the same timing references to determine the windows for initiating downlink traffic to Class B end-devices. End-devices of type Class C are listening when not transmitting. A Class C

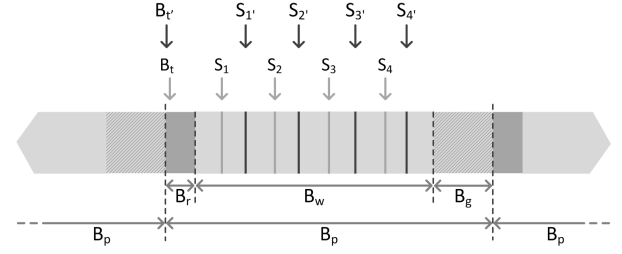


Figure 1: Beaconsing vulnerability

end-device consumes the most energy and is typically used for non battery-powered devices.

Beacons broadcasted by gateways for the Class B scheduled reception windows, are not protected by encryption or a digital signature. Without such protection, beacons are vulnerable to manipulation by eavesdropping and replay attacks. By manipulating timing references in a LoRaWAN, it is possible to desynchronize the additional Class B reception windows between end-devices and the network. This will cause a Denial-of-Service for Class-B downlink traffic between the network and end-devices.

Figure 1 shows a sequence of beacon periods (B_p), of which one beacon period is provided with more detail. A beacon period contains a beacon reserved window (B_r) in which beacons can be received, a beacon window (B_w) in which the reception windows are scheduled, and a beacon guard (B_g) which is used for finishing transmissions so that these will not collide with beacons in the next (B_r) window. The figure also includes an example of the scheduling of authentic and manipulated reception windows. (B_t) represents a timing reference in an authentic beacon that is received in (B_r). The markers (S_1) to (S_4) represent reception windows that are based on (B_t). (B_t') represents a timing reference in a manipulated beacon that is received in (B_r), just before the authentic beacon. The markers (S_1') to (S_4') represent reception windows that are based on the manipulated (B_t') timing reference. When the end-device uses the manipulated beacon, its calculated Class B reception windows will differ from the windows calculated by the network. The network is now unable to initiate Class B downlink traffic towards the end-device until a next authentic beacon is processed.

The fact that beacons are not encrypted, which allows attackers to build and send malicious beacons, has also been reported briefly by Yang [16] for the LoRaWAN v1.0 specification but not in the context of DoS attacks, and in a white-paper by Miller [12]. Our analysis confirms that this vulnerability is still present in the LoRaWAN v1.1 specification.

This vulnerability may be mitigated in future LoRaWAN specifications by adding authentication to beacons, for instance by signing beacons. However, this requires a (shared) key between the network and all end-devices. Other generic network control communications could also benefit from such a shared key.

3.2 Downlink routing vulnerability

LoRaWAN provides downlink traffic options for all end-device classes. When the network needs to initiate downlink traffic towards an end-device, it uses the gateway which was last used by the end-device for its uplink transmission. The network server keeps

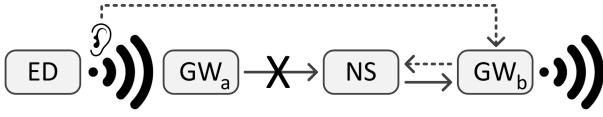


Figure 2: Downlink routing vulnerability

track of the last used gateway for each end-device in its downlink routing path database. This is specifically described for Class B end-devices, but must be applicable to Class A and Class C end-devices as well since these also need to use the correct gateway for downlink transmissions.

End-devices broadcast their data during uplink communication. The data is received by all gateways in reach, that all forward the data to the network server. The network server will process the data, keep track of the used gateway (downlink routing path database), and perform de-duplication if data comes in from multiple gateways. When the network server needs to send data to a specific end-device, the network server looks for the gateway to use in its downlink routing path database, and uses that gateway to broadcast the data. Because that gateway was also used by the end-device, it is expected the end-device is still in reach and able to receive the transmission. When the network uses a wrong gateway for downlink traffic, then the transmission may not be received by the end-device if it is out of reach. Since there is no standard packet confirmation (like TCP/IP has) in the LoRaWAN protocol, the downlink transfer of data can fail without being known by the network.

Whenever it is possible to manipulate the network server's downlink routing path database, then it is possible to impact the availability of specific end-devices for downlink traffic. Since the routing path database is updated based on the signal quality indicators of the last uplink of the end-device, it is possible to manipulate the routing path database by using a combination of eavesdropping and replay attacks.

Figure 2 shows how an end-device (ED) initiates an uplink transmission. Normally this uplink transmission is received by a gateway (GW_a) in reach, which forwards the packet to the network server (NS) for further processing. In this example, the uplink transmission is eavesdropped by a malicious party, and replayed via a gateway (GW_b) that is out-of-reach for the end-device. Eavesdropping LoRaWAN traffic is straightforward as the LoRaWAN specification is readily available, required hardware is cheap and widely available, and different examples of source code (gateway implementations) are available for download¹²³⁴. When the transmission via GW_b is received by the network server first, it updates the downlink routing path database based on GW_b , and it will discard the authentic packet via GW_a because it is a duplicate packet. The next time the network initiates a downlink transmission to the end-device, it will use GW_b . Since GW_b is out-of-reach of the end-device, the downlink transmission will never reach the end-device.

Aras et al. reported that this vulnerability in the LoRaWAN v1.0 specification allows a wormhole attack, in which a malicious device captures the packets from an end-device and transmits them to

another distant end-device to replay the captured packets [3]. However, the possibility to exploit this vulnerability in a DoS attack by modifying the downlink routing path table, has not been mentioned before. Furthermore, our analysis confirms that this vulnerability is still present in the LoRaWAN v1.1 specification.

Directions for mitigating this vulnerability must be found in using a handshake between network server and end-device to verify that the end-device is reachable after updating the downlink routing path table, and restoring the previous path in case the end-device has become unreachable. The end-device confirmation message must include details of the new gateway, and include a digital signature (MIC).

3.3 Join-Accept replay vulnerability

Before an end-device can be active in a LoRaWAN, it must join the network first. There are two methods for an end-device to join a LoRaWAN: Activation-by-Personalization (ABP) or Over-the-Air-Activation (OTAA). When using ABP, all configuration settings must be provided by the network server, and manually configured on the end-device. OTAA offers a more flexible solution. After storing only a few configuration details on the end-device, the OTAA procedure automatically exchanges the required configuration settings between network and end-device. The OTAA procedure can be run as many times as required (initially, re-join after power loss, joining in case of roaming, et cetera).

During OTAA, details are exchanged (using Join-Request and Join-Accept messages), and security contexts are generated. The generated security contexts make sure that both the end-device and the network are using the correct security keys for encryption and message signing. The Join-Accept message however is vulnerable to an eavesdropping and replay attack combination.

The first step in OTAA is that the end-device transmits a Join-Request message to join the network. Such a message contains an AppEUI, DevEUI, and DevNonce. The AppEUI and DevEUI are unique addresses in the EUI-64 address space. The DevNonce is a random value which is used to prevent replay attacks. The Join-Request message is processed by the network server. If the network server accepts the end-device, a Join-Accept message is generated and returned to the end-device. The Join-Accept message contains details (such as AppNonce and NetID) that are used for generating a correct security context (NwkSKey, AppSKey, frame counters, et cetera) so that the end-device uses the same security keys as the network. Unfortunately, the Join-Accept message does not include a reference to the Join-Request message that it was generated for, and therefor OTAA is vulnerable for replaying older Join-Accept messages addressed to the same end-device.

This vulnerability has been described for the LoRaWAN specification v1.0 by Zulian in October 2016 [17] as well as by Avoine and Ferreira in July 2017 [4]. Our analysis confirms that this vulnerability still is present in the LoRaWAN specification v1.0.2, but has been successfully mitigated in v1.1 by using an incrementing JoinNonce value to prevent replay attacks of Join-Accept messages. However, LoRaWAN specification v1.1 includes a compatibility mode for v1.0.2-only end-devices. For these devices, this vulnerability is still present, even though the network is running on v1.1.

¹<https://github.com/brocaar/lora-gateway-bridge>

²<https://github.com/lab11/LoraWAN-Gateway>

³<https://github.com/Lora-net/LoRaMac-node>

⁴<https://github.com/things4u/ESP-1ch-Gateway-v4.0>

4 ATTACK MODELING AND VALIDATION

During our analysis, we also used a more formal approach by constructing a CPN model for each attack. CPN is a formal modeling language which offers both a formal and graphical notation for representing systems or processes in a compact way. CPNs are commonly used in protocol analysis (e.g., [6][7][14]). The graphical notation is based on rectangles that represent transitions (for example events), and circles that represent places (for example a state or condition). The rectangles and circles are connected by arrows that form pre- and post-conditions for transitions, and form the *flow relation*. A CPN contains one or more tokens, which (when certain conditions are met) enable transitions. When an enabled transition is fired, it consumes one or more tokens, and possibly produces one or more tokens.

We used CPNtools to simulate our CPN models. In addition, CPNtools offers state space analysis to validate the functional correctness of the models. We analyzed the simulation results using the graphical feedback provided by the tool, and by log monitors that log output for specific places and transitions. These CPN models can be used to simulate the vulnerabilities and test possible fixes and/or improvements. Our CPN models are available on GitHub⁵.

4.1 Beaconsing vulnerability

We constructed the high-level CPN shown in figure 3 to model the beaconsing vulnerability, based on the LoRaWAN specification. The model contains a gateway, an attacker, and an end-device. The gateway broadcasts beacons in a specific interval. The attacker broadcasts additional malicious beacons or replaces authentic beacons by broadcasting malicious beacons at the exact same moment with a higher signal strength. All malicious beacons broadcasted by the attacker contain a manipulated timing reference. The end-device receives and processes the beacons, calculates the beacon reception window (B_r), makes sure that only one beacon is processed in each beacon receive window, and finally calculates the additional reception windows.

We used CPNtools to simulate the model. In the following, we summarize the most important simulation details. First, the gateway sends out two beaconsing network frames:

```
(RFU,Time,CRC,(Antenna,(Lat,Long)),RFU,CRC) @ time-unit
(0,128000,"", (0,(002001,038100)),0,"") @ 128081
(0,256000,"", (0,(002001,038100)),0,"") @ 256176
```

The attacker inserts a manipulated beaconsing frame just before the authentic frame, containing a slightly different timing reference (256001 instead of 256000):

```
(0,256001,"", (0,(009999,009999)),0,"") @ 256001
```

The end-device receives all three beaconsing frames. In the first beaconsing reception window (B_r), the first authentic beaconsing frame is received and processed. In the second beaconsing reception window, the malicious beaconsing frame is accepted and processed by the end-device because it is received before the authentic beaconsing frame. The example shows that besides a manipulated timing reference, also the location details of the beacon can be manipulated.

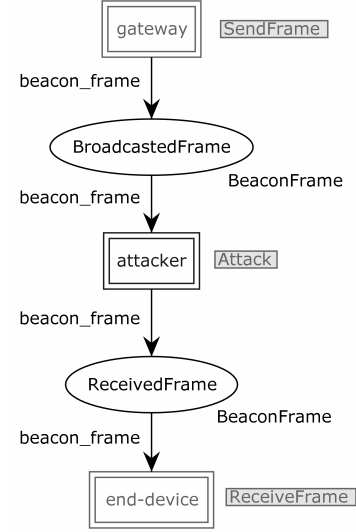


Figure 3: Beaconsing vulnerability CPN

Based on the beaconsing frames the below scheduled Class B reception windows are calculated. The calculated reception windows for timing reference 256000 are only included as a reference since that beaconsing frame is rejected by the end-device. In the second beaconsing reception window, the network server uses the additional Class B reception windows related to timing reference 256000, while the end-device is using the additional Class B reception windows related to timing reference 256001.

Time-ref	Slot1	Slot2	Slot3	Slot4
128000 >>	[130379, 161099, 191819, 222539]			
256000 >>	[258380, 289100, 319820, 350540]			
256001 >>	[258637, 289357, 320077, 350797]			

4.2 Downlink routing vulnerability

We constructed the high-level CPN shown in figure 4 to model the downlink routing vulnerability, based on the LoRaWAN specification. The model contains an end-device, an attacker, two gateways, and a network server. The end-device sends uplink traffic in a specific interval. The attacker eavesdrops all traffic and replays it using a different network path. The gateways forward the packets without changes. The network server validates the security context of the incoming packets, accepts valid packets, and updates the downlink routing path database according to the used gateway.

We used CPNtools to simulate the model. In the following, we summarize the most important simulation details. First, the end-device sends out two uplink network packets.

```
(MHDR,DevAddr,FCtrl,FCnt,FPort,FRMPayload,MIC) @ time-unit
(0x40,0x63190126,0x00,1,0x01,0x914C787A,...) @ 0
(0x40,0x63190126,0x00,2,0x01,0x914C787A,...) @ 10000
```

These two uplink network packets are eavesdropped by the attacker, and replayed using a different network path. Now there are two gateways that are receiving and forwarding exactly the same two uplink network packets. The result is that four uplink network packets are received by the network server. The order

⁵<https://github.com/E-v-Es/Denial-of-Service-Attacks-on-LoRaWAN/>

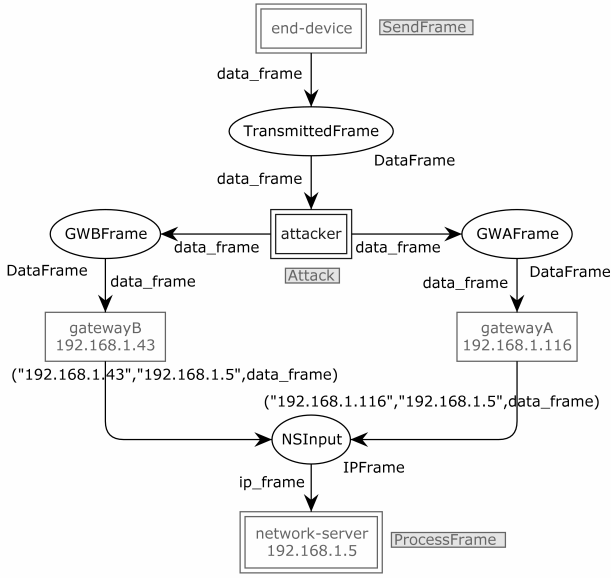


Figure 4: Downlink routing vulnerability CPN

is depending on the time of transmission, and the speed of the used network links. Based on frame counters and de-duplication, only the first network packets with a specific frame counter will be accepted by the network server.

```
(GW-IP,NS-IP,(LoRa-frame)) @ time-unit
(192.168.1.116,192.168.1.5,(0x40,0x63...,1,...)) @ 32
(192.168.1.43,192.168.1.5,(0x40,0x63...,1,...)) @ 167
(192.168.1.43,192.168.1.5,(0x40,0x63...,2,...)) @ 10067
(192.168.1.116,192.168.1.5,(0x40,0x63...,2,...)) @ 10302
```

Based on the accepted network packets, the downlink routing path database is updated. Since one of the replayed network packets arrives before the authentic packet, it is accepted by the network server, and the downlink routing path databases is updated with the IP address (192.168.1.43) of the gateway used by the attacker.

```
DevAddr -> GW-IP @ time-unit
0x63190126 -> 192.168.1.116 @ 32
0x63190126 -> 192.168.1.43 @ 10067
```

The first line added to the downlink routing path database directs downlink traffic to the correct gateway. That gateway is used by the end-device for uplink traffic, and it is able to deliver downlink traffic because the end-device is in reach. The second line added to the downlink routing path database sets the gateway for downlink traffic to the gateway used by the attacker. This gateway is not in reach of the end-device, and therefore downlink traffic will never reach its destination.

4.3 Join-Accept replay vulnerability

We constructed the high-level CPN shown in figure 5 to model the Join-Accept replay vulnerability, based on the LoRaWAN specification. The model contains an end-device, an attacker, a gateway, and a network server. The end-device joins the network using OTAA, and sends uplink traffic, including application of a security context. The gateway simply forwards traffic between end-device and

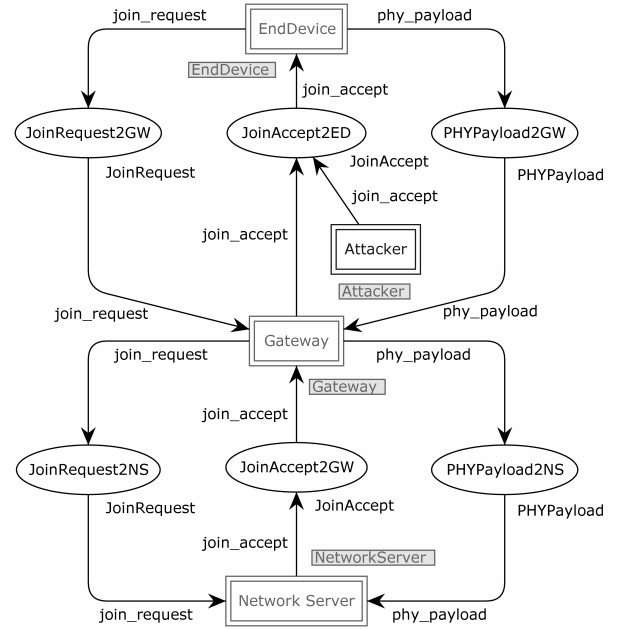


Figure 5: End-device activation vulnerability CPN

network server. The network server accepts the end-device during OTAA, manages the security contexts, and processes uplink traffic from the end-device. The attacker performs a replay attack on the Join-Accept message.

We used CPNtools to simulate the model. In the following, we summarize the most important simulation details. First, the end-device sends out a Join-Request.

```
(MHDR,AppEUI,DevEUI,DevNonce,MIC) @ time-unit
(0x00,0xBB6100F...,0xCF3662E8...,0x0101,0x455...) @ 500000
```

The Join-Request message is received by the network server. When accepted, the network server generates a Join-Accept message which is returned to the end-device.

```
(AppNonce,NetID,DevAddr,DLSettings,RxDelay,CFList) @ time
(0x29229,0x130000,0x63190126,"","") @ 500918
```

Before the authentic Join-Accept message from the network server arrives at the end-device, and attacker has replayed an older Join-Accept message which is received by the end-device before the authentic Join-Accept message.

```
(AppNonce,NetID,DevAddr,DLSettings,RxDelay,CFList) @ time
(0x54321,0x130000,0x63190126,"","") @ 500004
```

From all uplink data packets transmitted by the end-device, and received by the network server, it can be seen that the packets before the replay attack are accepted and the packets are rejected from the replay attack onwards. Communication between end-device and network server is failing until the end-device successfully initiates a new OTAA.

```
(MHDR,DevAddr,FCtrl,FCnt,FPort,FRMPayload,MIC) @ time-unit
(0x40,0x6319...,0x00,8,0x01,0x914C7...,0x015...) @ 480820
(0x40,0x6319...,0x00,0,0x01,0x914C7...,0x015...) @ 540820
(0x40,0x6319...,0x00,1,0x01,0x914C7...,0x015...) @ 600820
...
```

5 CONCLUSIONS

We identified three vulnerabilities in the LoRaWAN protocol specification that can be used for launching DoS attacks against end-devices in a LoRaWAN network. We validated that these vulnerabilities can be exploited for DoS attacks by creating and simulating CPN models of relevant parts of the LoRaWAN protocol.

REFERENCES

- [1] LoRa Alliance. -. (-). <https://www.lora-alliance.org/what-is-lora>
- [2] LoRa Alliance. 2017. LoRaWAN v1.1 Regional Parameters. (2017). <https://www.lora-alliance.org/>
- [3] Emekcan Aras, Gowri Sankar Ramachandran, Piers Lawrence, and Danny Hughes. 2017. Exploring the security vulnerabilities of LoRa. In *IEEE International Conference on Cybernetics (CYBCONF)*. <https://doi.org/10.1109/CYBCONF.2017.7985777>
- [4] Gildas Avoine and Loïc Ferreira. 2016. Rescuing LoRaWAN 1.0. (2016). <https://eprint.iacr.org/2017/651>
- [5] CPNtools. -. (-). <http://cpntools.org>
- [6] Ole Martin Dahl and Stephen D Wolthusen. 2006. Modeling and Execution of Complex Attack Scenarios using Interval Timed Colored Petri Nets. In *IEEE International Workshop on Information Assurance (IWIA'06)*, pp.157–168. <https://doi.org/10.1109/IWIA.2006.17>
- [7] Jorge C. A. De Figueiredo and Lars M. Kristensen. 1999. Using Coloured Petri Nets to Investigate Behavioural and Performance Issues of TCP Protocols. In *Workshop on Practical Use of Coloured Petri Nets and Design/CPN*. Aarhus University, 21–40.
- [8] LoRa Alliance. 2015. LoRaWAN Specification v1.0. (2015). <https://www.lora-alliance.org/>
- [9] LoRa Alliance. 2016. LoRaWAN Specification v1.0.2. (2016). <https://www.lora-alliance.org/>
- [10] LoRa Alliance. 2016. LoRaWAN v1.0 Regional Parameters. (2016). <https://www.lora-alliance.org/>
- [11] LoRa Alliance. 2017. LoRaWAN v1.1 Specification. (2017). <https://www.lora-alliance.org/>
- [12] Robert Miller. 2015. LoRa Security - Building a Secure LoRa Solution. (2015). <https://labs.mwrinfosecurity.com/assets/BlogFiles/mwri-LoRa-security-guide-1.2-2016-03-22.pdf>
- [13] National Institute of Standards and Technology (NIST). 2001. Announcing the advanced encryption standard (AES). (2001), 8–12 pages. [https://doi.org/10.1016/S1353-4858\(10\)70006-4](https://doi.org/10.1016/S1353-4858(10)70006-4)
- [14] Gonzalez Nieto, Suratosé Tritilanunt, Colin Boyd, Ernest Foo, and Juan Manuel Gonz. 2006. Using Coloured Petri Nets to Simulate DoS-resistant protocols. (2006). <https://eprints.qut.edu.au/23982/>
- [15] Semtech. -. (-). <https://www.semtech.com/technology/lora/what-is-lora>
- [16] Xueying Yang. 2017. LoRaWAN: Vulnerability Analysis and Practical Exploitation (MSc thesis, Delft University of Technology). (2017).
- [17] Simone Zulian. 2016. Security threat analysis and countermeasures for LoRaWANTM join procedure (MSc thesis, University of Padova). (2016). http://tesi.cab.unipd.it/53210/1/zulian_simone_tesi.pdf